# Planstudio v2: implementation specs for lower-cost models

Addendum to `v2-kernel-plan.md`, written 2026-11-09.

The main plan says **what** to build and why. This addendum says **exactly how**, in pieces small enough for a lower-cost model (for example DeepSeek V4, or a Haiku- or Sonnet-class model) to implement without making design decisions. Every design decision is made here. An implementing model fills in code against these specs and proves it with the listed tests.

I haven't evaluated DeepSeek V4. Run the trial in §2.4 before relying on any model for a whole phase.

---

## 1. How to use this document

- **Humans:** pick the next task card in §11 whose dependencies are done, paste the card plus the sections it names into the agent's context, and review the result with the checklist in §12.
- **Implementing agents:** read §3 (rules) and the sections your card names. Do nothing else.
- **Spec owner:** a stronger model or a human. Writes and changes this document, reviews Tier A cards, and decides anything marked **DECISION**.

---

## 2. Model roles and workflow

### 2.1 Tiers

| Tier | Who | Does |
|---|---|---|
| **A: spec and review** | A strong model (Opus or Fable class) or the maintainer | Owns this document, writes public interfaces and tests first, reviews geometry and solver code, decides open points |
| **B: implement** | A mid-cost model (DeepSeek V4, Sonnet class) | Implements cards with a given algorithm, interfaces and tests |
| **C: mechanical** | A cheap model (Haiku class) | Fixtures, docs, renames, test data, formatting, simple glue |

Each card in §11 names its tier. A Tier B model can attempt a Tier A card only if a Tier A reviewer checks the result.

### 2.2 Test-first loop (for every card)

1. The interfaces and tests named in the card exist, written by the spec owner or a previous card. If they don't, stop and report.
2. Implement until `cargo test -p <crate>` passes, and `cargo clippy -- -D warnings` and `cargo fmt --check` are clean.
3. Don't edit existing tests or public signatures unless the card says so.
4. Commit with the card id in the message (`P1-06: axis solver spanning tree`).
5. Report back: what changed, the test output, anything unclear. Don't guess.

### 2.3 Context pack per card

Keep context small. Cheaper models degrade with long contexts. Give the agent:

- the card;
- §3 rules;
- the spec sections the card lists;
- the files the card lists, in full;
- nothing else (no whole repo dumps).

### 2.4 Trial before committing to a model

Run cards P0-01, P1-01, P1-02 and P1-06 with the candidate model. Record for each: did it pass the tests without help, how many retries, how many review comments, tokens and cost. If P1-06 (the solver) needs more than two review rounds, keep Tier A on solver and geometry cards and use the cheaper model for the rest.

---

## 3. Rules for implementing agents (copy into the new repo's `AGENTS.md`)

1. Work only on the card you were given. Touch only the files it lists, plus new files inside the module it names.
2. Never change a public type, function signature, test, fixture or this document unless the card says so. If they look wrong, stop and report.
3. No new dependencies unless the card lists them. Never add network calls.
4. **Units:** all lengths are `i64` millimetres in stored data. Use `f64` only inside solver and geometry computations, and round with `round_half_even` when storing.
5. **Coordinates:** x grows east, y grows **south** (screen and SVG convention). North is −y. Never use "left/right" in user-facing text; use compass directions.
6. No `unwrap`, `expect` or `panic!` in library code. Return `Result<_, Diagnostic>` or `Vec<Diagnostic>`. `#![forbid(unsafe_code)]` in every crate except the OpenSKP FFI crate.
7. All user-facing messages come from `diagnostics::message(code, args)` (§8) so wording stays consistent.
8. Deterministic output: sort by id or name wherever order is visible (JSON, DSL, SVG). Use `BTreeMap`, not `HashMap`, in serialized types.
9. Every public function has a doc comment saying units and coordinate convention when relevant.
10. If a test cannot pass without breaking a rule, stop and report the conflict.

---

## 4. Repository skeleton (card P0-01 creates it)

```
Cargo.toml                 # workspace, edition 2024, license GPL-3.0-or-later
AGENTS.md                  # §3
crates/
  core/                    # no I/O
    src/lib.rs
    src/units.rs           # Mm, Source, tolerance defaults (§5.1)
    src/intent.rs          # intent document types (§5.2)
    src/resolved.rs        # nodes, walls, openings, rooms after solving (§5.3)
    src/dsl/{mod.rs, lexer.rs, parser.rs, printer.rs}   # §7
    src/solve/{mod.rs, graph.rs, axis.rs, report.rs}    # §6
    src/build/{mod.rs, walls.rs, openings.rs, faces.rs} # §9
    src/diagnostics.rs     # §8
    src/commands.rs        # §10
  render/                  # Phase 2
  checks/                  # Phase 3
  server/                  # Phase 4
  skp/                     # Phase 2, OpenSKP FFI
  editor/                  # Phase 5
  cli/                     # Phase 1
  import-v1/               # Phase 7
vendor/                    # pinned third-party code, each with LICENSE and SOURCE.txt
(commit hash)
fixtures/
  lgh154-a.plan            # §7.5
  lgh154-b.plan
  lgh154-c.plan
  lgh154-a.expected.json
  ...
```

Dependencies allowed in `core` for Phase 1: `serde`, `serde_json`, `thiserror`, `nalgebra` (solver), `strsim` (name suggestions). Dev only: `proptest`, `insta`, `pretty_assertions`.

---

## 5. Data model

### 5.1 Units, sources and tolerances

```
        pub type Mm = i64;

        #[derive(Clone, Copy, Debug, PartialEq, Eq, PartialOrd, Ord, Serialize,
Deserialize)]
        #[serde(rename_all = "snake_case")]
        pub enum Source { Laser, Tape, Printed, Drawing, Estimated, Assumed }

        impl Source {
            /// Default tolerance in mm when the statement gives none.
            pub fn default_tolerance(self) -> Mm {
                match self {
                    Source::Laser => 3, Source::Tape => 5, Source::Printed => 20,
                    Source::Drawing => 30, Source::Estimated => 50, Source::Assumed => 10,
                }
            }
        }
```

A relation with no source (`align`, `open`) is **exact**: tolerance 0, and the solver never changes it.

### 5.2 Intent document (the source of truth)

**DECISION (made): the Phase 1 intent is room-first.** Rooms are axis-aligned rectangles given by clear size, placed relative to each other. Walls are generated from the rooms. Free-standing walls and angled walls come in a later DSL version (§13).

```rust
pub struct Plan {
    pub name: String,
    pub default_wall: Quantity,          // interior wall thickness, e.g. 90 assumed
    pub default_outer_wall: Quantity,    // e.g. 300 assumed
    pub rooms: BTreeMap<RoomName, Room>, // declaration order kept in `order`
    pub order: Vec<RoomName>,
    pub relations: Vec<Relation>,
    pub measures: Vec<Measure>,
    pub openings: Vec<Opening>,
    pub wall_overrides: Vec<WallOverride>,
}

pub struct Quantity { pub value: Mm, pub source: Source, pub tolerance: Mm }

pub struct Room {
    pub name: RoomName,                  // unique, case-sensitive, [A-Za-zÅÄÖåäö0-
9_]+
    pub width: Option<Quantity>,         // east–west clear size; None = "?"
(unknown)
    pub depth: Option<Quantity>,         // north–south clear size
    pub apartment: Option<String>,
    pub phase: Phase,                    // existing | demolish | new
}

pub enum Side { North, East, South, West }
pub struct FaceRef { pub room: RoomName, pub side: Side }   // the room's inner
face on that side

pub enum Relation {
    /// `subject` lies east of `of`; gap = wall thickness between subject.west and
of.east
    EastOf  { subject: RoomName, of: RoomName, gap: Gap },
    WestOf  { subject: RoomName, of: RoomName, gap: Gap },
    SouthOf { subject: RoomName, of: RoomName, gap: Gap },
    NorthOf { subject: RoomName, of: RoomName, gap: Gap },
    /// Faces on the same side are collinear (exact).
    Align   { side: Side, rooms: Vec<RoomName> },
}

pub enum Gap {
    Wall(Quantity),   // `gap 90` or `gap 121 source drawing`; plain `gap` uses
default_wall
    Open,             // `open`: no wall, the two rooms form one space
}

/// A direct measurement between two parallel faces, e.g. a laser reading across two
rooms.
pub struct Measure {
    pub from: FaceRef, pub to: FaceRef,
    pub value: Quantity,
    pub reference: bool,  // true = checked and reported, never drives geometry
}

pub struct Opening {
    pub kind: OpeningKind,           // Door | Window | Passage
    pub face: FaceRef,               // the room side it sits in
    pub at: Quantity,                // offset along the face, see §9.3 for the
```

```
origin rule
            pub width: Quantity,
            pub height: Option<Mm>,
            pub hinge: Option<Side>,         // doors: compass side of the jamb with the
hinge
            pub opens_into: Option<RoomName>,// doors: which room the leaf swings into
            pub phase: Phase,
        }

        pub struct WallOverride { pub face: FaceRef, pub thickness: Quantity }  // e.g.
outer wall measured 160
```

### 5.3 Resolved model (computed, never edited directly)

```
        pub struct Resolved {
            pub rooms: BTreeMap<RoomName, RoomRect>,  // solved inner faces
            pub nodes: BTreeMap<NodeId, Pt>,
            pub walls: BTreeMap<WallId, Wall>,
            pub openings: BTreeMap<OpeningId, ResolvedOpening>,
            pub spaces: Vec<Space>,                    // rooms joined by `open`
            pub diagnostics: Vec<Diagnostic>,
        }
        pub struct Pt { pub x: Mm, pub y: Mm }
        pub struct RoomRect { pub west: Mm, pub east: Mm, pub north: Mm, pub south: Mm }
        pub enum RefSide { Left, Center, Right }  // relative to from→to; internal only
        pub struct Wall {
            pub id: WallId, pub from: NodeId, pub to: NodeId,
            pub thickness: Mm, pub reference: RefSide,
            pub kind: WallKind,                       // Interior | Outer
            pub between: (Option<FaceRef>, Option<FaceRef>), // rooms on either side
            pub thickness_source: Source,
        }
```

**Ids are semantic and stable:**

- Walls: `W:<RoomA>.<side>|<RoomB>.<side>` for interior walls (names sorted), `W:<Room>.<side>:outer` for outer walls. Add `#2` when one face has several segments.
- Nodes: `N:<x>,<y>`.
- Openings: `O:<Room>.<side>:<n>`, numbered by `at`.

---

## 6. Solver (Phase 1, deterministic, axis by axis)

**DECISION (made):**

- Phase 1 implements this graph solver itself. It is simple, deterministic, and explains conflicts as concrete chains of numbers, which is the main user value.
- SolidCraft's sketch solver (main plan) sits behind the same `Solver` trait for non-axis-aligned geometry later (card P1-14, after the Phase 0 spike).

### 6.1 Variables and equations

Solve x (east–west) and y (north–south) separately. For each room there are 4 face variables: `west`, `east` (x axis) and `north`, `south` (y axis).

Every statement becomes an **edge** `(a, b, value, tolerance, origin)` meaning `coord(b) − coord(a) = value`:

| Statement | Edge |
|---|---|
| room width `w` | `west → east = w` |
| room depth `d` | `north → south = d` |
| `S east_of O gap g` | `O.east → S.west = g` |
| `S west_of O gap g` | `S.east → O.west = g` |
| `S south_of O gap g` | `O.south → S.north = g` |
| `S north_of O gap g` | `S.south → O.north = g` |
| `... open` | same as gap with `value = 0, tolerance = 0` |
| `align north A B C` | `A.north → B.north = 0`, `A.north → C.north = 0`, tolerance 0 |
| `measure F to T = v` | `F → T = v` (faces must be on the same axis; else E010) |
| `measure ... reference` | not an edge; checked after solving (W020) |
| `?` size | no edge |

Plain `gap` uses `default_wall`. Exact edges (`align`, `open`) have tolerance 0. A `gap` with no source takes `default_wall`'s source and tolerance.

### 6.2 Algorithm per axis

1. **Build the graph.** Nodes are face variables on this axis, and edges come from §6.1. Store each edge both ways (reverse = negated value).
2. **Components.** Find connected components (union-find).
   - The component that contains the **anchor** (the first declared room's `west` on x, its `north` on y) is fixed at 0.
   - Every other component is **under-constrained**: emit W001 (§6.4) and anchor it at its own first face, so a drawing still appears.
3. **Spanning tree.** Breadth-first search from the component's anchor, visiting edges in declaration order. Assign `coord(b) = coord(a) + value`.
4. **Cycle check.** For every non-tree edge `(a, b, v)`:
   - `residual = v − (coord(b) − coord(a))`.
   - The cycle is the tree path from a to b plus this edge.
   - `cycle_tolerance` is the sum of the tolerances of all edges in the cycle.
   - If `|residual| > cycle_tolerance`, emit **E003 conflict** (§6.3).
   - Otherwise record the cycle for step 5.
5. **Distribute (weighted least squares).**
   - If there are no conflicts and at least one cycle within tolerance, solve the component's edges by weighted least squares.
   - Unknowns are the face coordinates, with the anchor fixed. Weight is `1 / max(tol,1)²`, and exact edges are hard constraints.

    - Use `nalgebra`: equality-constrained least squares via the KKT system.
    - Each edge's adjusted value is reported in the result so the user sees what absorbed the difference. Edges adjusted by more than half a millimetre are listed in W004.
6. **Round** every coordinate to whole mm (half-even).
7. If any E003 exists, the plan **does not resolve**. Return diagnostics and the last good resolved model, if there is one.

### 6.3 Conflict message (E003)

List the cycle as two chains between the same two faces. The chain without the non-tree edge is the "implied" side.

```
E003 Conflict on the north–south axis: 50 mm.
  Garderob depth 2360 (laser) + wall 90 (assumed) + Sovrum depth 2090 (laser) = 4540
  measure Garderob.north → Sovrum.south = 4490 (laser)
  Allowed difference: 19 mm.
  Fix one of: mark a measurement as reference, correct a value, or change the wall (90 → 40
would fit).
```

The structured fields (`axis`, `residual`, `tolerance`, `chain_a: [EdgeRef]`, `chain_b: [EdgeRef]`, `suggestions`) go in `Diagnostic.data` for MCP clients.

### 6.4 Missing-measurement report (W001)

For each under-constrained component, find the pair of faces, one in the anchored component and one in the loose one, that are parallel, face each other, and are closest. Suggest that measurement:

```
W001 Not tied east–west: Förråd. Measure from Hall.east to Förråd.west, or add "Förråd
east_of Hall gap 90".
```

Rooms with `?` sizes whose size is not implied by other edges also get W001: `Hall depth unknown; measure Hall.north to Hall.south`.

### 6.5 Solver trait

```
        pub trait Solver {
            fn solve(&self, plan: &Plan) -> SolveResult; // per-axis coords + adjusted edges
+ diagnostics
        }
        pub struct AxisGraphSolver;      // §6.2, Phase 1
        // pub struct SketchSolver;      // SolidCraft adapter, later (P1-14)
```

---

# 7. DSL

## 7.1 Grammar (EBNF)

```
file        = { line } ;
line        = [ statement ] [ comment ] NEWLINE ;
comment     = "#" { any } ;
statement   = plan | default | room | relation | align | measure | opening | wall ;
plan        = "plan" STRING ;
default     = ("default_wall" | "default_outer_wall") qty ;
room        = "room" NAME size "x" size [ src ] [ "apartment" STRING ] [ phase ] ;
size        = INT | "?" ;
relation    = INDENT? NAME? rel_kw NAME gapspec ;      (* indented lines belong to the last
room *)
rel_kw      = "east_of" | "west_of" | "south_of" | "north_of" ;
gapspec     = "open" | "gap" [ INT [ src ] ] ;
align       = INDENT? "align" side NAME { NAME } ;     (* indented: last room is first in
the list *)
measure     = "measure" face "to" face "=" INT [ src ] [ "reference" ] ;
opening     = ("door" | "window" | "passage") face "at" INT "width" INT [ "height" INT ]
              [ "hinge" side ] [ "opens" NAME ] [ src ] [ phase ] ;
wall        = "wall" face "thickness" INT [ src ] ;
face        = NAME "." side ;
side        = "north" | "east" | "south" | "west" ;
qty         = INT [ src ] ;
src         = "source" ( "laser" | "tape" | "printed" | "drawing" | "estimated" | "assumed"
)
              [ "tol" INT ] ;
phase       = "existing" | "demolish" | "new" ;
INT         = digit { digit } | digit { digit } "." digit { digit } "m" ;   (* "1.86m" =
1860 *)
NAME        = letter { letter | digit | "_" } ;       (* letters include åääöÅÄÖ *)
STRING      = '"' { any but '"' } '"' ;
```

Notes:

- Keywords are lower case. Room names are case-sensitive.
- `INDENT` is any leading whitespace. An indented relation without a subject applies to the last declared room.
- Lengths: plain integers are mm. `1.86m` is 1860. Anything else is E006.

## 7.2 Printer (canonical form)

`print(parse(s))` must equal the canonical form of `s`. `parse(print(p)) == p` for every valid plan (proptest).

Canonical form:

1. `plan`, then the defaults.
2. Rooms in declaration order. Each room's relations are indented by two spaces under it, in this order: east_of, west_of, south_of, north_of, then align.
3. Then `measure`, `wall`, `door`, `window`, `passage` lines, each group sorted by face then `at`.

Omit a `source` equal to the default (`assumed` for defaults, none for exact relations), and omit `tol` equal to the source default. Comments are not preserved in canonical form; the raw text is kept in storage separately.

### 7.3 Parse errors

E006 with line and column: `E006 Line 7, column 18: expected "x", found "*".`

### 7.4 Name resolution

Unknown room → E001 with the closest name by Jaro–Winkler (`strsim`), suggested if the score is at least 0.85: `E001 Unknown room "Sovrun" (line 9). Did you mean "Sovrum"?` A duplicate room name gives E002.

### 7.5 Fixtures (Phase 0 creates these files; numbers from the laser measurements in this repo's history)

**`fixtures/lgh154-a.plan`**: existing state, consistent:

```
plan "Gnejsen lgh 154, existing (laser 2026-11-07)"
default_wall 90 source assumed
default_outer_wall 300 source assumed

room Garderob 1860 x 2360 source laser
room Sovrum 1860 x 2090 source laser
  south_of Garderob gap
  align west Garderob
room Hall 1100 x ?
  east_of Garderob gap 121 source drawing
  align north Garderob
room WC 2370 x 990 source laser
  east_of Hall gap
  align north Hall
room Vardagsrum 3560 x 3480 source laser
  south_of Hall open
  south_of WC gap 123 source drawing
  align west Hall

window Sovrum.south at 380 width 1350 source laser
window Vardagsrum.south at 1610 width 1280 source laser
door WC.west at 240 width 450 hinge south opens Hall source estimated
passage Garderob.east at 0 width 680 source estimated
```

**`fixtures/lgh154-b.plan`**: fixture A plus the 4.49 m laser reading. Expect a conflict:

```
measure Garderob.north to Sovrum.south = 4490 source laser
```

**`fixtures/lgh154-c.plan`**: fixture A plus a straight facade. Expect a conflict just over tolerance:

```
align south Sovrum Vardagsrum
```

### 7.6 Expected results (put in `*.expected.json`; tests compare exactly unless a tolerance is given)

**Fixture A**, solved faces (x east, y south, anchor at Garderob west/north = 0):

| Room | west | east | north | south |
|---|---|---|---|---|
| Garderob | 0 | 1860 | 0 | 2360 |
| Sovrum | 0 | 1860 | 2450 | 4540 |
| Hall | 1981 | 3081 | 0 | 1113 |
| WC | 3171 | 5541 | 0 | 990 |
| Vardagsrum | 1981 | 5541 | 1113 | 4593 |

Also for fixture A:

- **Diagnostics:** exactly `W003` (facade step) and two `I010` (derived walls Garderob.east|Vardagsrum.west and Sovrum.east|Vardagsrum.west, both 121 mm), no errors.
- **W003 text:** `Facade not straight: Sovrum.south 4540 vs Vardagsrum.south 4593 (53 mm). Add "align south Sovrum Vardagsrum" if it is straight.`
- **Areas, m², 2 decimals:** Garderob 4.39, Sovrum 3.89, Hall 1.22, WC 2.35, Vardagsrum 12.39. Space {Hall, Vardagsrum} 13.61.
- **Interior walls that must exist:**
  - Garderob.south|Sovrum.north, 90 thick, x 0–1860;
  - Garderob.east|Hall.west, 121, y 0–1113;
  - Garderob.east|Vardagsrum.west, 121, y 1113–2450 (bridged across the end of the Garderob/Sovrum wall);
  - Sovrum.east|Vardagsrum.west, 121, y 2450–4593 (extended 53 mm past Sovrum's south face);
  - Hall.east|WC.west, 90, y 0–990;
  - Vardagsrum.north|WC.south, 123, x 3171–5541.
- **No wall** between Hall and Vardagsrum (`open`).
- **South facade:** the Sovrum.south outer wall continues east across the partition's end to x 1981, where the Vardagsrum.south outer wall (53 mm further south) begins. There is no notch.

**Fixture B**: E003 on the north–south axis:

- residual −50 (measured 4490, implied 4540), cycle tolerance 3 + 10 + 3 + 3 = 19;
- chain A: Garderob depth 2360, gap 90, Sovrum depth 2090; chain B: the measure;
- text exactly as in §6.3; not resolved.

**Fixture C**: E003 on the north–south axis, residual 53.

- Chain A: Garderob depth 2360 + gap 90 + Sovrum depth 2090 = 4540.
- Chain B: WC depth 990 + gap 123 + Vardagsrum depth 3480 = 4593, joined through `align north` (exact).
- Cycle tolerance: 3 + 10 + 3 + 3 + 30 + 3 = 52. Since 53 > 52, it is a conflict.

**Unit test for distribution (§6.2 step 5).** Faces P and Q with two paths:

- Path 1: one laser edge of 2000.

- Path 2: a laser edge of 1500 plus an estimated edge of 520 (tol 50).
- Residual 20, within the tolerance of 56.
- Expected adjusted values within ±1 mm: path 1 = 2000, laser 1500, estimated 500, Q − P = 2000.

**Fixture D (later, for multi-apartment work):** lgh 153 readings from 2026-11-08:

- bedroom 3064 × 4611, door 280 from its west wall;
- hall 5872 long, 1184 wide beside the WC and 2442 overall;
- WC 2008 × 1060 inside;
- stairwell wall to WC outer face 1600.

The spec owner writes it in DSL v2, once walls not bounding rooms exist.

---

## 8. Diagnostics

```
        pub struct Diagnostic {
            pub code: Code,                 // E001…, W001…, I001…
            pub severity: Severity,         // Error | Warning | Info
            pub message: String,            // from diagnostics::message, English, compass
terms
            pub span: Option<Span>,         // DSL line/col when applicable
            pub refs: Vec<String>,          // semantic ids involved (rooms, faces, walls)
            pub data: serde_json::Value,    // structured detail for MCP clients
        }
```

| Code | Severity | Meaning | Message template |
|---|---|---|---|
| E001 | Error | Unknown room | `Unknown room "{name}" (line {line}).{ Did you mean "{suggestion}"?}` |
| E002 | Error | Duplicate room | `Room "{name}" is declared twice (lines {a} and {b}).` |
| E003 | Error | Measurement conflict | §6.3 |
| E004 | Error | Opening outside its face | `{kind} on {face} at {at} width {width} runs past the face ({length} mm long).` |
| E005 | Error | Rooms overlap after solving | `{a} and {b} overlap by {dx} × {dy} mm.` |
| E006 | Error | Parse error | `Line {line}, column {col}: expected {expected}, found {found}.` |
| E007 | Error | Opening on an open side | `{face} has no wall ("open"); a {kind} needs a wall.` |
| E010 | Error | Measure between non-parallel faces | `{from} and {to} are not parallel; measure between two north/south or two east/west faces.` |
| W001 | Warning | Under-constrained | §6.4 |
| W003 | Warning | Facade step 1–100 mm | `Facade not straight: {a} {ya} vs {b} {yb} ({d} mm). Add "align {side} {ra} {rb}" if it is straight.` |
| W004 | Warning | Values adjusted to close a cycle | `Adjusted to fit within tolerance: {list of "edge old→new"}.` |
| W005 | Warning | Wall thinner than 70 mm or thicker than 600 mm | `Wall {id} is {t} mm thick; check the measurements around it.` |
| W020 | Warning | Reference measure differs | `Reference measure {from}→{to}: drawn {drawn}, measured {value} ({diff} mm).` |
| I010 | Info | Wall thickness derived, not stated | `Wall {id} is {t} mm, derived from the room positions.` |

---

# 9. Building the resolved model from solved rooms

## 9.1 Faces facing each other

Two room faces "face each other" if all of these hold:

- they are on opposite sides (one room's east face and another's west face, or north and south);
- the second face lies on the outward side of the first;
- the distance between them is 0–600 mm;
- their spans overlap by more than 0 mm.

## 9.2 Wall generation (line sweep per face)

For every room face:

1. **Split.** Split its span at the span ends of every facing face (§9.1).
2. **Classify each segment:**
    - Covered by a facing face through an explicit relation with `Gap::Wall(g)`: interior wall of thickness g, source from the relation.
    - Covered by a facing face through `Gap::Open`: no wall.

    - Covered by a facing face with no explicit relation: interior wall with the derived thickness (distance between faces), source `Assumed`, and I010.
    - Not covered: outer wall, thickness from a `wall` override on that face or else `default_outer_wall`. Its inner face is the room face, and it grows outward.
3. **Bridge.** An uncovered segment no longer than 600 mm that lies between two interior segments of the same thickness on the same face becomes interior with that thickness. Example: where the Garderob/Sovrum wall ends against the partition.
4. **Extend.** An uncovered segment no longer than 100 mm at the end of a face, next to an interior segment, extends that interior segment. Example: the 53 mm below Sovrum on Vardagsrum's west face.
5. **Corner continuation.** Outer walls on the same compass side are extended across the end of an interior wall between them, so the facade has no notch.
6. **Merge.** Merge collinear adjacent segments of the same kind, thickness and room pair.
7. **Deduplicate.** An interior wall is produced once, from the face that comes first in sort order.

**Geometry of a wall segment:**

- the node line is the room face it was generated from, so `RefSide` puts that face on the node line;
- `from` is the west or north end, `to` the east or south end;
- nodes are rounded to mm and shared when coordinates are equal.

**Facade step check (W003):** for each compass side, compare the inner faces of outer walls on that side whose spans are separated only by one interior wall. Emit W003 when the difference is 1–100 mm.

### 9.3 Openings

- `at` is measured along the inner face from its **west end** on north and south faces, and from its **north end** on east and west faces.
- The opening occupies `[at, at + width]` along the face.
- It must fit inside one wall segment on that face, otherwise E004. An open side gives E007.
- `hinge` must be one of the two compass directions along the face (west or east on a north face), otherwise E004 with the message "hinge must be west or east on a north face".

### 9.4 Wall polygons (faces.rs) for rendering

- Each wall is a rectangle: node line plus thickness on its `RefSide`.
- **L corner** (two walls at a node, perpendicular): extend both rectangles to the outer corner, so the outer faces meet. The overlap is drawn once.
- **T** (a wall ending on another wall's face): extend the ending wall to the through wall's near face. No extension into it.
- **Straight continuation with different thickness:** no extension; the step is visible.

**Tests:**

- An L corner of 90 and 300 mm walls produces a closed outline.
- Every room's inner rectangle is exactly its solved faces.
- The union of all wall polygons plus room rectangles has no gaps for fixture A (the rasterised check in P1-12).

### 9.5 Areas

- Room area is width × depth from solved faces, in m² rounded to 2 decimals for display. Store exact mm².
- Spaces are connected components of rooms joined by `open`, with area as the sum.

---

## 10. Commands (Phase 1 core, used by CLI, MCP and editor)

```
        pub enum Command {
            ApplyDsl { patch: String },                          // statements to add or
replace (§10.1)
            SetSize { room: RoomName, width: Option<Quantity>, depth: Option<Quantity> },
            AddRelation(Relation),
            RemoveRelation { index: usize },
            AddMeasure(Measure),
            SetReference { measure: usize, reference: bool },
            AddOpening(Opening),
            RemoveOpening { id: OpeningId },
            SetWall(WallOverride),
            RenameRoom { from: RoomName, to: RoomName },         // updates all references
        }
        pub struct ApplyResult {
            pub ok: bool,                 // false if any Error; plan unchanged in that case
            pub plan: Option<Plan>,       // new plan when ok
            pub resolved: Option<Resolved>,
            pub diff: Vec<Change>,        // rooms/walls/openings added, removed, moved
(with mm deltas)
            pub diagnostics: Vec<Diagnostic>,
        }
        pub fn apply(plan: &Plan, commands: &[Command], dry_run: bool) -> ApplyResult;
```

**Rules:**

- All-or-nothing: if any command fails or any Error diagnostic results, return `ok: false` and leave the plan unchanged.
- `dry_run` computes everything and changes nothing.

### 10.1 Patch semantics

A DSL patch is parsed as statements. Each statement is matched against the plan by a key, and replaces the matching statement or is added if there's none:

| Statement | Key |
|---|---|
| room | name |
| relation | (subject, kind, of) |
| align | (side, first room) |
| measure | (from, to) |
| opening | (face, at) |
| wall | face |

A line `- <statement>` removes the matching statement.

---

## 11. Task cards

**Format:**

```
ID · Title · Tier · Size (S ≤ 1 session, M 1–3, L 3–6) · Depends on
Read: spec sections · Files: create/modify
Do: …
Done when: tests …
```

### Phase 0: setup, fixtures, spikes

| ID | Card | Tier | Size | Depends |
|---|---|---|---|---|
| P0-01 | Workspace skeleton (§4), `AGENTS.md` (§3), CI running fmt, clippy and test for native and wasm32 | C | S | — |
| P0-02 | Write fixtures A–C and expected JSON exactly as §7.5–7.6 | C | S | P0-01 |
| P0-03 | **Spike:** model fixture A's walls in SolidCraft's sketch solver (vendored); report whether faces and thickness map cleanly, and how conflicts are reported | A | M | P0-01 |
| P0-04 | **Spike:** BenchUp `inference.rs` snapping on a 2D plane in a 50-line egui demo | B | M | P0-01 |
| P0-05 | **Spike:** write a 4-wall `.skp` with OpenSKP (C++ via `cc`) and open it in SketchUp for Web; report version and result | B | M | P0-01 |
| P0-06 | **Spike in v1:** add room and wall labels to v1's `plan_view` output, pre-sized for Claude; check Claude names walls correctly on the basplan | B | M | — |

### Phase 1: kernel and CLI (core crate)

| ID | Card | Tier | Size | Depends | Done when |
|---|---|---|---|---|---|
| P1-01 | Types §5.1–5.3 with serde; `Source::default_tolerance` | C | S | P0-01 | Serde round-trip tests for every type |
| P1-02 | Lexer (§7.1 tokens, `1.86m`, comments, spans) | B | S | P1-01 | Token tests including Swedish letters and E006 positions |
| P1-03 | Parser to `Plan` (§7.1, indentation rule) | B | M | P1-02 | Fixtures A–C parse; 15 error cases give exact E006 text |
| P1-04 | Name resolution, E001 suggestions, E002 | C | S | P1-03 | `Sovrun` gives the §7.4 text |
| P1-05 | Printer and canonical form (§7.2); proptest round-trip | B | M | P1-03 | `parse(print(p)) == p` over 1000 generated plans |
| P1-06 | Graph build and spanning-tree solve (§6.1, §6.2 steps 1–3, 6) | B | M | P1-04 | Fixture A faces equal §7.6 table |
| P1-07 | Cycle check and E003 text (§6.2 step 4, §6.3) | B | M | P1-06 | Fixtures B and C give exactly the specified diagnostics |
| P1-08 | Weighted least-squares distribution and W004 (§6.2 step 5) | A | M | P1-07 | Distribution unit test passes within ±1 mm; no change to fixture A |
| P1-09 | Under-constraint and missing-measurement W001 (§6.4) | B | M | P1-06 | New fixture with an unconnected room gives the W001 text |
| P1-10 | Wall generation line sweep (§9.1–9.2) and W003, I010 | A | L | P1-06 | Fixture A walls and diagnostics as §7.6 |
| P1-11 | Openings (§9.3), E004, E007 | B | M | P1-10 | Fixture A has 4 openings in the right walls; three error fixtures |
| P1-12 | Wall polygons and corners (§9.4); rasterised gap check test | A | M | P1-10 | §9.4 tests |
| P1-13 | Commands and patches (§10); all-or-nothing; diff | B | M | P1-05, P1-11 | Adding the fixture B line through a patch returns ok=false with E003 and leaves the plan unchanged |
| P1-14 | `SketchSolver` adapter behind the `Solver` trait (only if P0-03 says go) | A | L | P0-03, P1-08 | Fixture A gives the same faces as `AxisGraphSolver` |
| P1-15 | CLI: `plan check FILE`, `plan solve FILE --json`, `plan fmt FILE` | C | S | P1-13 | Snapshot tests (`insta`) for fixtures A–C |
| P1-16 | Acceptance: fixtures A–C end to end through the CLI | C | S | P1-15 | All expected JSON matches |

### Phase 2: render and export (outline; the spec owner details before starting)

- **P2-01 (B):** scene primitives from `Resolved` (wall polygons, room labels, dimension lines, openings with swings). SVG backend with layers `walls`, `openings`, `labels`, `dims`, `grid`.
- **P2-02 (B):** labelled render (set-of-marks).
  - Every wall, room and opening carries its semantic id as a visible label.
  - Labels never overlap, using greedy placement with fallback leader lines.
- **P2-03 (B):** PNG via `resvg`, pre-sized with Anthropic's resize rule (main plan, "Giving the model vision"). Metadata: mm-per-pixel, origin, image size.
- **P2-04 (B):** crop by id or mm rectangle at a target token budget.
- **P2-05 (B):** PDF via `svg2pdf`, with A3/A4, scale, title block and before/after sheets.

- **P2-06 (B):** DXF via the `dxf` crate.
- **P2-07 (A review):** `.skp` via OpenSKP, with groups per room and tags per phase.

### Phase 3: checks, rooms, diagnostic views (outline)

- **P3-01 (B):** port v1 permit checks.
  - Read the v1 sources `lib/plan/domain/checks.ts` and `structure.ts`.
  - The spec owner first lists each rule with inputs, threshold and message.
- **P3-02 (B):** conflict view, colouring walls by the largest residual on any edge through their faces.
- **P3-03 (B):** missing-measurement view, drawing suggested laser shots as arrows between the faces from W001.
- **P3-04 (B):** diff view, with added walls green, removed red and moved amber, plus labels with mm deltas.
- **P3-05 (B):** overlay on a source image with a calibration matrix.

### Phase 4: MCP server (outline; tool schemas below are fixed now)

```
// apply
{ "plan_id": "string", "patch": "DSL text", "dry_run": false }
// → { ok, diff[], diagnostics[], summary }

// get
{ "plan_id": "string", "view": "summary|dsl|json", "fields":
["rooms","walls","openings","diagnostics"] }

// check
{ "plan_id": "string" }
// → { diagnostics[], next_steps[] }   (next_steps from W001, E003, open findings)

// render
{ "plan_id": "string",
  "view": "plan|labels|conflicts|missing|diff|overlay",
  "region": { "id": "W:…" } | { "rect_mm": [x,y,w,h] },
  "max_tokens": 1600 }
// → image + { mm_per_px, origin_mm, size_px }

// find
{ "plan_id": "string", "query": "room or wall name, partial ok" }

// import
{ "plan_id": "string", "kind": "photo|sketch|v1|dxf|skp", "file_id": "string", "note": "free
text" }
// → proposed DSL patch + questions[]

// history
{ "plan_id": "string", "limit": 20 }
```

- **P4-01 (B):** `rmcp` server skeleton plus the tools above over `core`.
- **P4-02 (B):** storage as a command log plus snapshots in SQLite.
- **P4-03 (A):** `import` for photos.
  - Structured-output schema for laser readings.
  - Labelled render alongside, so the model proposes which faces each reading spans.

    - Questions when ambiguous.
- **P4-04 (B):** MCP App "tap the wall".
- **P4-05 (B):** eval harness and the task set. The spec owner writes 20 tasks with pass criteria first.
- **P4b (B):** auth provider integration, chosen by the spec owner.

### Phases 5–7

The spec owner writes card lists before each phase starts, following the same format. The editor (Phase 5) cards each map one SketchUp tool to `Command`s from §10. Push/Pull on a room face, for example, becomes `SetSize`, plus a relation gap change when it's a shared wall.

---

## 12. Review checklist (Tier A or maintainer, per card)

- [ ] Only the listed files changed; no test or signature edited without permission.
- [ ] `cargo fmt --check`, `clippy -D warnings` and tests pass natively and on wasm32 (core).
- [ ] No `unwrap`, `expect` or `panic` in library code; errors are diagnostics with codes from §8.
- [ ] Integer mm in stored types; coordinates y-south; compass words in messages.
- [ ] Output order deterministic.
- [ ] New behaviour has a test that fails without the change.
- [ ] Messages match §8 templates exactly.
- [ ] For geometry and solver cards: one hand-checked example in the PR description, with the numbers worked out.

---

## 13. Deferred to DSL v2 (spec owner, before Phase 5)

- Free-standing walls and partitions not bounding a room on both sides: `wall from Hall.east@300 to …`.
- Non-rectangular rooms (L shapes as two rectangles joined by `open` already work in v1).
- Angled walls (needs `SketchSolver`).
- Multi-apartment plans with stairwells (fixture D).
- Phases on rooms and walls in the intent (before/after sheets): v1 of the DSL has `phase` on rooms and openings only.

---

## 14. Cost note for lower-cost models

The main plan's estimates assume Claude prices. Moving Tier B and C cards to a cheaper model lowers agent spend roughly in proportion to its price. Check the price of the model you use; I haven't verified DeepSeek V4's.

Keep these on Tier A, even when the rest moves:

- P0-03, the solver spike;
- P1-08, P1-10, P1-12 and P1-14, the solver and geometry cards;
- P2-07 (`.skp` review) and P4-03 (photo import);

- all spec writing.

They are about 10–15% of the sessions, but most of the risk.

The eval gate in the main plan is separate. It tests which models can **use** the finished MCP tools well (Opus, Sonnet, Haiku), not which model **builds** the system.